EMINES
School of Industrial Management

UNIVERSITÉ MOHAMMED VI
POLYTECHNIQUE

Les bases de la programmation en C

CPI1A – Semestre 1

MOHAMMED VI
POLYTECHNIC
UNIVERSITY

Plan du cours

- Nous allons passer de l'algorithmique à un langage de haut niveau qui peut être compilé pour donner une application.
- La syntaxe du langage sera vue progressivement, nous allons dans un premier temps donner la structure d'un programme.
- En abordant en premier lieu la fonction principale main d’un programme c
- Par la suite, nous allons traiter les fonctions standards d’entrées et sorties via les fonctions printf et scanf
- Les types pré-définis
- Les opérateurs
- Les instructions de branchement
- Etc.

Comment programmer ?

Résoudre un problème

- Penser son code (raisonnement logique et intuitif) Détailler les étapes complètes du programme à concevoir - développer l'algorithme correspondant
- Implémenter (contrôler et commenter chaque étape) Ecrire (traduire) l'algorithme dans le langage de programmation choisi (C/C++)
- Compiler / Exécuter : Transformer le code en un programme exécutable par l'ordinateur
- Debugger si nécessaire (retour au point 1)

La compilation

Le C est un langage *compilé :* Cela signifie qu'un programme C est décrit par un fichier texte, appelé *fichier source.*

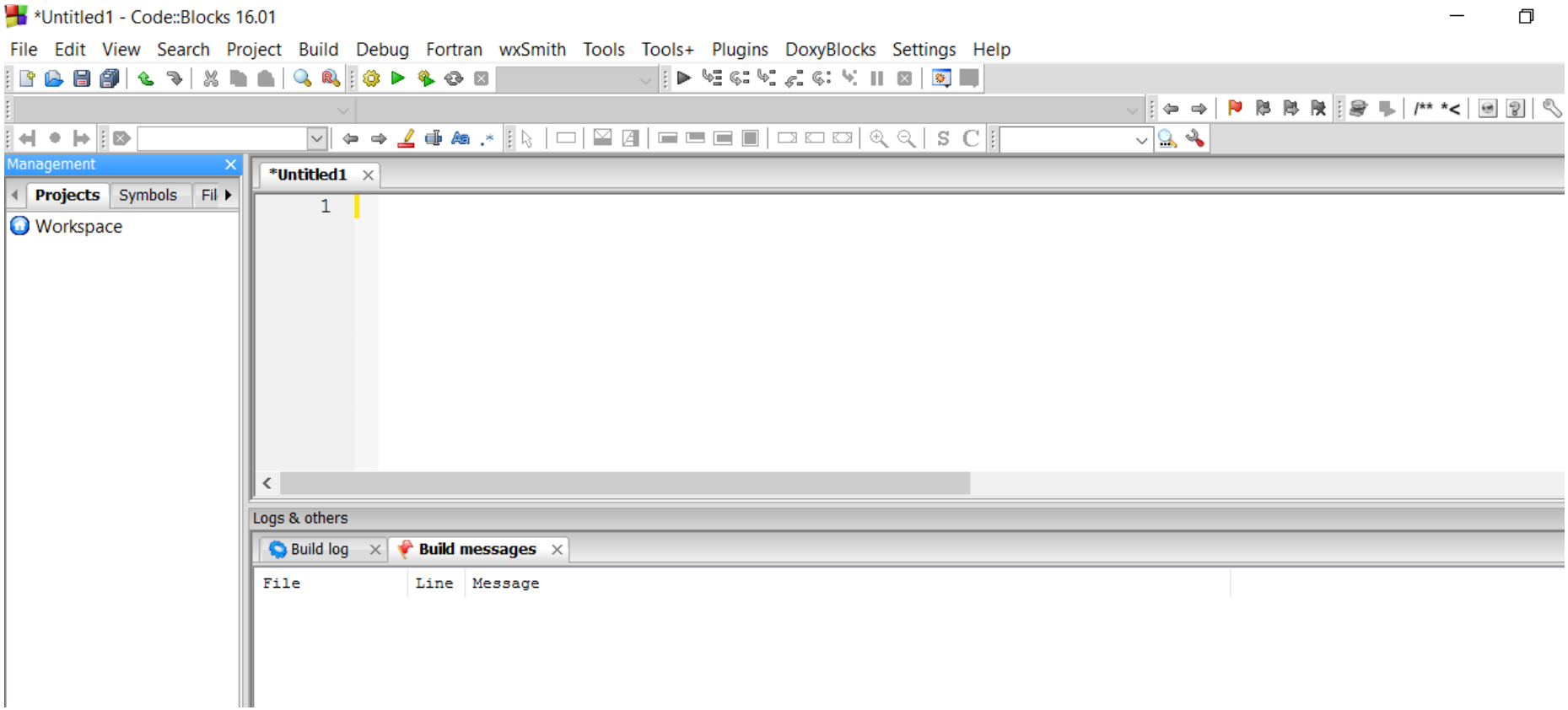
Ce fichier n'etant évidemment pas exécutable par le microprocesseur, il faut le traduire en langage machine via un *compilateur :*

Syntaxe de compilation d'un programme C :

gcc [*options] fichier.c [-llibrairies]*

EMINES
School of Industrial Management
UNIVERSITÉ MOHAMMED VI POLYTECHNIQUE

Code::Blocks



La notion d'Expression

Une *expression est une suite de composants élémentaires syntaxiquement correcte, par* exemple :

x = 2 ou bien (i >= 0) && (i < 10) && (p[i] != 0)

La notion d'Instruction

Une *instruction est une expression suivie d'un point-virgule qui signifie* « *évaluer cette expression* », Plusieurs instructions peuvent être rassemblées par des accolades *{ et }*

par exemple :

```
if (x != 0) {
        z = y / x;
        t = y % x;
}
```

```
int a;   //une instruction qui a pour objectif de déclarer la variable a
 int b = 1, c;
```

Structure d'un programme C

```
/* des inclusions de fichiers en-tete (fonctions dans des bibliothèques) */
#include <nom_bibliotheque>


/* des définitions de variables et constantes */
definition de constante_1 ;
...
definition de constante_M ;
definition de variable_1 ;
...
definition de variable_N ;

/* des fonctions */
definition de la fonction 1
{
   corps de la fonction 1
}
...

definition de la fonction P
{
   corps de la fonction P
}


/* partie principale */
definition de la fonction principale
{
  corps de la fonction principale
}
```

Structure d'un programme C

En C, toute variable doit faire l'objet d'une déclaration avant d'être utilisée.

Un programme C se présente de la façon suivante :

[*directives au préprocesseur]*
[*déclarations de variables externes]*
[*fonctions secondaires]*
[*fonction principale]*

Les bibliothèques

Utiliser des bibliothèque

indiquer quelles sont les bibliothèques utilisées par le programme.

Utilisé par l'expression :

#include <nom-de-fichier>

Exemple : #include <stdio.h> //bibliothéque des entrées et sorties

Les bibliothèques

Utilisé par l'expression :

La gestion des fichiers (**stdio.h**) /* Entrees-sorties standard */

Les fonctions mathématiques (**math.h**)

Traitement de chaînes de caractères (**string.h**)

Le traitement de caractères (**ctype.h**)

Utilitaires généraux (**stdlib.h**)

Date et heure (**time.h**)

Les composants élémentaires du C

composants élémentaires suivants :

- Les identificateurs : de donner un nom à une entité du programme (nom d'une variable ou de fonction, un type pré défini, etc.)
- Les mots-clefs : sont réservés pour le langage lui-même (if, else, const, return, while, etc.)
- Les commentaires : Un commentaire débute par /* et se termine par */.

EMINES
School of Industrial Management
UNIVERSITÉ MOHAMMED VI POLYTECHNIQUE

Remarque préliminaire : commenter le code !

Remarque préliminaire :

commenter le code → on ajoute du texte pour expliquer à quoi servent les lignes de code.

```
#include <stdio.h>
/* Programme pour calculer la moyenne
à partir des notes saisies des etudiants
*/
int main() {
    int nbEtudiant; //Variable pour stocker le nombre des étudiants
    printf("Veuillez préciser le nombre des étudiants"); //demander à l'utilisate
    scanf("%d",&nbEtudiant); //lire le nomnbre des étduaints
    for (i=0;i<nbEtudiant;i++) {
        //saisir la note
        scanf("%d",&note);
    }
}
```

Les types prédéfinis

toute variable est d'un type précis.

La mémoire de l'ordinateur se décompose en une suite continue d'octets. Chaque octet de la mémoire est caractérisé par son *adresse.*

Les types de base en C concernent les caractères, les entiers et les flottants (nombres réels).

Ils sont désignés par les mots-clefs suivants :

Char int float double short long unsigned

Le type caractère

Le mot-clef char désigne un objet de type caractère.

char est codé sur un octet ; c'est l'objet le plus élémentaire en C.

c+1 est valide et désigne le caractère suivant dans le code ASCII.

Exemple :

```
int main() {
        char c = 'A';
         printf("%c", c +1 );
}
```

Les types entiers

Le mot-clef désignant le type entier est int.

Le type int peut être précédé d'un attribut de précision (short ou long) et aussi et/ou d'un attribut de représentation (unsigned)

En général :

- Un short int est codé sur 16 bits.
- Un objet de type long int a au moins la taille d'un int.

Les types entiers

En C, on utilisera un des types suivants en fonction de la taille des données à stocker :

Pour récupérer la taille allouée pour un type, la syntaxe en C est :

taille = sizeof(unsigned short);

signed char	$[-2^7; 2^7[$
unsigned char	$[0; 2^8[$
short int	$[-2^{15}; 2^{15}[$
unsigned short int	$[0; 2^{16}[$
int	$[-2^{31}; 2^{31}[$
unsigned int	$[0; 2^{32}[$
long int	$[-2^{63}; 2^{63}[$
unsigned long int	$[0; 2^{64}[$

Les types flottants

Les types float, double et long double servent à représenter des nombres en virgule flottante.

`float`	32 bits	flottant
`double`	64 bits	flottant double précision
`long double`	64 bits	flottant quadruple précision

Déclaration de variables

Chaque variable doit avoir un nom et un type.

type nom de variable;

l'instruction de délcalration de variable est comme suit

int NBEtudiant;

int note, valeur;

float note, moyenne;

char caractére;

Les constantes

Une constante est une valeur qui apparaît littéralement dans le code source d'un programme,

Elles ne peuvent pas changer de valeur au cours du programme.

Avec le commande #define !

Exemple : */* Les constantes symboliques définies par l'utilisateur */*

```
#define MAX 100
#define PI 3.14
#define LANGAGE "C"
```

Les constantes entières & Réelles

```
#define TVA 20.6
#define DIV 100
int main(void){
    printf("10 euros HT = %.2lf TTC\n", 10*TVA/DIV + 10);
    printf("20 euros HT = %.2lf TTC\n", 20*TVA/DIV + 20);
    printf("30 euros HT = %.2lf TTC\n", 30*TVA/DIV + 30);
    return 0;
}
```

Les constantes caractères

le mettre entre apostrophes (par ex. ’A’ ou ’$’).

Les seuls caractères imprimables qu’on ne peut pas représenter de cette façon sont l’antislash et l’apostrophe, qui sont respectivement désignés par \\ et \’.

les caractères non-imprimables les plus fréquents

`\n`	nouvelle ligne	`\r`	retour chariot
`\t`	tabulation horizontale	`\f`	saut de page
`\v`	tabulation verticale	`\a`	signal d’alerte
`\b`	retour arrière		

EMINES
School of Industrial Management
UNIVERSITÉ MOHAMMED VI POLYTECHNIQUE

Les opérateurs

L'affectation	Les opérateurs arithmétiques
variable = expression	+ addition - soustraction * multiplication / division % reste de la division (modulo)

Les opérateurs relationnels	Les opérateurs logiques booléens
> strictement supérieur >= supérieur ou égal < strictement inférieur <= inférieur ou égal == égal != différent	&& et logique || ou logique ! négation logique

Les opérateurs

Les opérateurs d'affectation composée	Incrémentation et de Décrémentation
Les opérateurs d'affectation composée sont : += -= *= /= %= &= ^= |= <<= >>= Pour tout opérateur *op, l'expression* *expression-1 op= expression-2* est équivalente à *expression-1 = expression-1 op expression-2*	int a = 3, b, c; b = ++a; /* a et b valent 4 */ c = b++; /* c vaut 4 et b vaut 5 */

Les opérateurs

L'opérateur de conversion de type	L'opérateur adresse

L'opérateur de conversion de type, appelé *cast, permet de modifier explicitement le type* d'un objet. On écrit :
(type) objet
Par exemple :
main() {
int i = 3, j = 2;
printf("%f \n",(float)i/j);
}
	L'opérateur d'adresse & appliqué à une variable retourne l'adresse-m´emoire de cette variable.
	La syntaxe est :
	&objet

La fonction principale main

La fonction principale main est la seule à être appelée lors de l'exécution du programme C.

Le fichier C contiendra donc toujours une fonction main,

```
int  main()
{
   déclarations de variables internes
   instructions
}
```

Ecrire une programme qui permet d'échanger les valeurs de deux variables a et b de type int et float respectivement.

La fonction d'écriture printf

La fonction printf est une fonction d'impression formatée, ce qui signifie que les données sont converties selon le format particulier choisi.

Equivalent à l'instruction ecrire().

Sa syntaxe est :

printf("*chaîne de contrôle ",expression-1, ..., expression-n);*

Les formules d'impression printf

format	conversion en	écriture
%d	int	décimale signée
%ld	long int	décimale signée
%u	unsigned int	décimale non signée
%lu	unsigned long int	décimale non signée
%o	unsigned int	octale non signée
%lo	unsigned long int	octale non signée
%x	unsigned int	hexadécimale non signée
%lx	unsigned long int	hexadécimale non signée
%f	double	décimale virgule fixe
%lf	long double	décimale virgule fixe
%e	double	décimale notation exponentielle
%le	long double	décimale notation exponentielle
%g	double	décimale, représentation la plus courte parmi %f et %e
%lg	long double	décimale, représentation la plus courte parmi %lf et %le
%c	unsigned char	caractère
%s	char*	chaîne de caractères

La fonction de saisie scanf

La fonction scanf permet de saisir des données au clavier et de les stocker aux adresses spécifiées par les arguments de la fonctions.

Equivalent à l'instruction lire();

scanf("*chaîne de contrôle ",expression-1, ..., expression-n);*

Les formats de saisie pour scanf

format	type d'objet pointé	représentation de la donnée saisie
%d	int	décimale signée
%hd	short int	décimale signée
%ld	long int	décimale signée
%u	unsigned int	décimale non signée
%hu	unsigned short int	décimale non signée
%lu	unsigned long int	décimale non signée
%o	int	octale
%ho	short int	octale
%lo	long int	octale
%x	int	hexadécimale
%hx	short int	hexadécimale
%lx	long int	hexadécimale
%f	float	flottante virgule fixe
%lf	double	flottante virgule fixe
%Lf	long double	flottante virgule fixe
%e	float	flottante notation exponentielle
%le	double	flottante notation exponentielle
%Le	long double	flottante notation exponentielle
%g	float	flottante virgule fixe ou notation exponentielle
%lg	double	flottante virgule fixe ou notation exponentielle
%Lg	long double	flottante virgule fixe ou notation exponentielle
%c	char	caractère
%s	char*	chaîne de caractères

Exercice

EMINES
School of Industrial Management

UNIVERSITÉ MOHAMMED VI
POLYTECHNIQUE

Partie II

Les instructions de Branchement

Les instructions de branchement

Branchement conditionnel if---else :

if (*expression-1)*

instruction-1

else if (*expression-2)*

instruction-2

...

else if (*expression-n)*

instruction-n

else

instruction-1

Créer un programme C qui lit une variable age de type int. SI
Age> 18 ecrire(« vous etes majeur »)
Sinon ecrire(« vous etes mineur »)

Branchement multiple switch

Sa forme la plus générale est celle-ci :

switch (*expression*) {
case *constante-1:*
liste d'instructions 1

case *constante-2:*
liste d'instructions 2

case *constante-n:*
liste d'instructions n

default:
liste d'instructions 1

}

Changer le programme age

Si age<2 ecrire(« bebe »)
6 ans : gamin
12 : jeune
16 : ado
18 : adulte
68 : papy
Sinon
Aucune phrase ne correspond

Branchement multiple switch

```
switch (age)
{
case 2:
  printf("Salut bebe !");

case 6:
  printf("Salut gamin !");

case 12:
  printf("Salut jeune !");

case 16:
  printf("Salut ado !");

default:
  printf("Je n'ai aucune phrase de prete pour ton age  ");

}
```

Les boucles

Boucle while :

while (*expression)*

instruction

Boucle do---while

do

instruction

while (*expression);*

Les boucles

Boucle for

for (*expr 1 ; expr 2 ; expr 3)*

Instruction

Exemple :

```
for (i = 0; i < 10; i++)
  printf("\n i = %d",i);
```

Créer un programme qui demande un nombre à l'utilisteur et affiche la suite de 1 à ce nombre

Exercice